```
#%% md
# Políticas de Seguridad y Gestión de Usuarios

## 1. Principios Generales de Seguridad

Estas políticas se basan en el principio de **Mínimo
Privilegio (Least Privilege)**, asegurando que los
usuarios y las aplicaciones solo tengan los permisos
estrictamente necesarios para realizar sus funciones.

| Principio | Descripción |
| :--- | :--- |
| **Separación de Funciones** | Los roles de
administración de base de datos (DBA) y los roles de
aplicación (App User) deben estar separados. |
| **Autenticación Fuerte** | Uso de contraseñas complejas
, rotación periódica y, si es posible, autenticación
multifactor (MFA). |
| **Cifrado en Tránsito y Reposo** | Uso de SSL/TLS para
todas las conexiones de red y cifrado de datos en el
disco (TDE o cifrado a nivel de sistema operativo). |

## 2. Políticas de Seguridad y Gestión de Usuarios (SQL -
General)

Aplicable a la base de datos `Collins_Demand_DB` (SQLite
, o su equivalente en un motor de producción como
PostgreSQL/MySQL).

### 2.1. Roles de Usuario SQL

| Rol | Propósito | Permisos (CRUD) |
| :--- | :--- | :--- |
| **DBA_Admin** | Mantenimiento, respaldo y gestión de
usuarios. | ALL (Todos los permisos) |
| **App_Writer** | Aplicación de carga de datos (e.g.,
ETL, scripts de actualización). | `ventas_historicas`:
CREATE, READ, UPDATE, DELETE. `pronosticos_activos`:
```

```
CREATE, READ, UPDATE, DELETE. |
| **App_Reader** | Aplicación de dashboard y reportes (e.g
., `app_dashboard.py`). | `ventas_historicas`: READ. `
pronosticos_activos`: READ. |
| **Analista_Datos** | Acceso de solo lectura para
análisis ad-hoc. | `ventas_historicas`: READ. `
pronosticos_activos`: READ. |

### 2.2. Ejemplo de Script de Creación de Rol (Conceptual
 - PostgreSQL/MySQL)

```sql
-- Creación de roles
CREATE ROLE App_Writer NOLOGIN;
CREATE ROLE App_Reader NOLOGIN;

-- Asignación de permisos a App_Writer
GRANT SELECT, INSERT, UPDATE, DELETE ON ventas_historicas
 TO App_Writer;
GRANT SELECT, INSERT, UPDATE, DELETE ON
pronosticos_activos TO App_Writer;

-- Asignación de permisos a App_Reader
GRANT SELECT ON ventas_historicas TO App_Reader;
GRANT SELECT ON pronosticos_activos TO App_Reader;

-- Creación de un usuario de aplicación y asignación de
rol
CREATE USER user_app_etl WITH PASSWORD '
contraseña_segura_etl';
GRANT App_Writer TO user_app_etl;
```

3. Políticas de Seguridad y Gestión de Usuarios (NoSQL
 - MongoDB)

Aplicable a la base de datos NoSQL (asumiendo **MongoDB
**).
```

3.1. Listado de Colecciones NoSQL

Se propone la siguiente estructura de colecciones para replicar la funcionalidad del sistema SQL, aprovechando la flexibilidad de MongoDB para almacenar documentos anidados.

Colección	Propósito	Estructura de Documento (Ejemplo)
historical_sales	Almacena los registros históricos de ventas.	`{ _id: <ObjectId>, producto: "A", cliente: "C1", fecha: <Date>, pedido_piezas: 150 }`
active_forecasts	Almacena los resultados de los pronósticos de demanda.	`{ _id: <ObjectId>, producto: "A", cliente: "C1", cluster: 3, fecha_pronostico: <Date>, ensemble: 160.5, min: 140, max: 180 }`
user_profiles	Almacena información de usuarios y roles para la aplicación.	`{ _id: <ObjectId>, username: "analista", role: "reader", last_login: <Date> }`

3.2. Roles de Usuario MongoDB

MongoDB utiliza un modelo de control de acceso basado en roles (RBAC).

Rol	Base de Datos/Colección	Permisos (Acciones)
dbAdmin	`*` (Todas las bases de datos)	`dbAdmin`, `userAdmin` (Gestión de la base de datos y usuarios)
appWriterRole	`demand_db` / `historical_sales`, `active_forecasts`	`find`, `insert`, `update`, `remove`
appReaderRole	`demand_db` / `historical_sales`, `active_forecasts`	`find` (Solo lectura)

3.3. Ejemplo de Script de Creación de Rol y Usuario (

MongoDB Shell)

```javascript
// 1. Conectarse a la base de datos de destino (demand_db)
use demand_db

// 2. Crear el rol de Escritura para la aplicación
db.createRole(
   {
      role: "appWriterRole",
      privileges: [
         { resource: { db: "demand_db", collection: "
historical_sales" }, actions: [ "find", "insert", "update
", "remove" ] },
         { resource: { db: "demand_db", collection: "
active_forecasts" }, actions: [ "find", "insert", "update
", "remove" ] }
      ],
      roles: []
   }
)

// 3. Crear el rol de Lectura para la aplicación/analistas
db.createRole(
   {
      role: "appReaderRole",
      privileges: [
         { resource: { db: "demand_db", collection: "" },
actions: [ "find" ] } // "" aplica a todas las
colecciones en la DB
      ],
      roles: []
   }
)

// 4. Crear un usuario y asignarle el rol de Escritura
db.createUser(
   {
```

```
        user: "user_mongo_etl",
        pwd: "contraseña_segura_mongo_etl",
        roles: [ { role: "appWriterRole", db: "demand_db
" } ]
    }
)
```